

Testing Laravel in Docker

Check the three containers, then test login, tokens, and logout.

Learning Goals

- Check that all three containers are healthy before testing the API.
- Test the public, login, token, protected-write, and logout cases with `curl` and the test page.
- Find the cause of a failure with `docker compose ps` and `docker compose logs`.

Check the Containers First

```
docker compose ps
```

NAME	SERVICE	STATUS
...-nginx-1	nginx	Up (healthy)
...-php-1	php	Up (healthy)
...-mysql-1	mysql	Up (healthy)

- All three must be **healthy** before you test the API.
- If a service is missing or unhealthy, read its log: `docker compose logs mysql php nginx`.

- All `curl` commands below are also in `test1-5-docker.sh`. Run it with:

```
bash test1-5-docker.sh
```

- The server is already running. You do **not** start anything with `php artisan serve`.
- The URL is `http://localhost:8000` (Module 2 used port 8080).
- There are four cases to test for every protected route: **no token**, **invalid token**, **valid token**, and **logged out**.

Test: Public Route

```
docker compose exec php php artisan route:list --path=api  
curl -H 'Accept: application/json' http://localhost:8000/api/students/1
```

- Reading students works without a token.
- The request travels Nginx → PHP-FPM → MySQL and back.
- Always send `Accept: application/json`. Without it, Laravel may try to redirect an unauthenticated request to a login page instead of returning JSON.

Test: Log In

```
curl -X POST http://localhost:8000/api/login \  
  -H 'Accept: application/json' -H 'Content-Type: application/json' \  
  -d '{"email":"student@university.edu","password":"student123"}'
```

```
{"message":"Login successful","token":"1|Xk3v9...",  
  "token_type":"Bearer","expires_in":3600,"user":"Student"}
```

A wrong password returns 401 .

Save the token in a variable, so you do not copy it by hand:

```
TOKEN="1|Xk3v9..." # paste the token from the login response
```

`test1-5-docker.sh` does this automatically. It reads the token with PHP **inside the container**, so your computer needs no PHP:

```
| docker compose exec -T php php -r 'echo json_decode(stream_get_contents(STDIN))->token;'
```

Test: Missing, Invalid, and Valid Token

```
curl -i -H 'Accept: application/json' \  
http://localhost:8000/api/me
```

```
curl -i -H 'Accept: application/json' \  
-H 'Authorization: Bearer invalid-token' \  
http://localhost:8000/api/me
```

```
curl -i -H 'Accept: application/json' \  
-H "Authorization: Bearer $TOKEN" \  
http://localhost:8000/api/me
```

- The first two return **401**. The last one returns **200** and the user.

Test: Protected Write

```
curl -i -X POST http://localhost:8000/api/students \  
  -H 'Accept: application/json' -H 'Content-Type: application/json' \  
  -d '{"name":"Grace Lee","email":"grace.lee@university.edu","age":20,"major":"Computer Science","year":2}'
```

- Without a token: **401**. Add `-H "Authorization: Bearer $TOKEN"` and the same request returns **201**.
- In Lesson 4, this request worked without any token. Now it does not.

Look Inside the Token Table

The database runs in the `mysql` container, so the query runs there:

```
docker compose exec -T mysql mysql -u ase230 -pase230pass studentdb -e \  
"SELECT id, tokenable_id, name, LEFT(token,12) AS token_hash FROM personal_access_tokens"
```

id	tokenable_id	name	token_hash
1	1	api-token	8c1f4e9a02b7

- The `token` column holds a hash. It does **not** match the string you sent.
- Each login creates a new row.

Log Out

```
curl -X POST http://localhost:8000/api/logout \  
  -H 'Accept: application/json' -H "Authorization: Bearer $TOKEN"  
  
curl -i -H 'Accept: application/json' \  
  -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/me
```

- The second request returns **401**: the token row was deleted, so the same string is no longer valid.
- The token is revoked on the server. Module 1's fixed tokens could never be revoked.

Status Codes at a Glance

Request	Status
GET /api/students/1 (public)	200
POST /api/login , wrong password	401
GET /api/me , no or invalid token	401
GET /api/me , valid Bearer token	200
POST /api/students , no token	401
POST /api/students , valid token	201
GET /api/me after POST /api/logout	401

If your results differ, the first place to look is `docker compose logs php`.

Test from the Browser: run1-5-tests

Typing `curl` commands is slow. Open a web page that runs the tests for you.

```
open run1-5-tests/index.html          # macOS  
# WSL2: open the file from Windows Explorer
```

- It is the same page as in Module 2. Its default **API Base URL** is `http://localhost:8000`.
- It calls the API with `fetch()` from your browser, and Nginx answers.
- Enter the email and password at the top. The default is the `student` demo user.

What the Tests Check

Part	Test	What it checks
1	Connection, wrong password, login	200 , then 401 , then a token that looks like id secret
2	GET /api/me	no token: 401 ; invalid token: 401 ; valid token: 200 and the user
3	Students	reads are public; create without a token is 401 , with a token is 201
3	Update, delete, validation	200 , then 404 after delete; bad input gives 422
		200 , then the user is i

When Something Fails

Symptom	Likely cause	What to do
port is already allocated	Another container uses port 8000	Stop the previous example, or change "8000:80"
Browser: 502 Bad Gateway	Nginx cannot reach PHP-FPM	docker compose ps , then docker compose logs php
Service never healthy	MySQL or PHP did not start	docker compose logs mysql php nginx
		The script now skips

Stop, Restart, and Keep Your Data

```
docker compose down  
docker compose up -d --wait  
bash test1-5-docker.sh
```

- After `down` and `up`, the students and demo users are still there: the `mysql-data` volume was kept.
- Old tokens still work until they expire (one hour) or you log out.
- `docker compose down -v` deletes the volume. Run the script again to rebuild the data.

Apply This to Your Project

`run1-5-docker.sh` copies the lesson files, so it would overwrite same-named files of your project. For Project 2, do not run it:

1. Put your Laravel app in `student-api/` (no `vendor/`, no local `.env`).
2. Copy `.env.docker` to `student-api/.env`.
3. `docker compose up -d --wait`
4. `docker compose exec php composer install`, then `key:generate` and `migrate --force`.
5. Test all four cases for each protected route, through `http://localhost:8000`.

The container uses PHP 8.4. Keep your dependencies compatible.

Command Summary

Students should remember:

Command	Meaning
<code>docker compose ps</code>	Are the three services healthy?
<code>docker compose logs php mysql nginx</code>	Why did a service fail?
<code>bash test1-5-docker.sh</code>	Run all curl tests
<code>docker compose exec -T mysql mysql -u ase230 -p... studentdb</code>	Look inside the database
<code>docker compose down</code>	Stop; keep the data
	Stop and delete the